

LatterFix

Decentralized Payroll & Task Escrow System

Product Requirements Document (PRD)

Document ID LFLF-PRD-001

Version 1.1.0

Release Date July 11, 2026

Author LatterFix Core Product Team

Status Approved / Production-Ready

1. Executive Summary

LatterFix is a modern decentralized Web3 application designed to solve trust, settlement

delay, and high fee issues inherent in traditional payroll systems and freelance work networks. By utilizing the **Stellar Soroban smart contract framework**, LatterFix replaces third-party intermediates with self-executing escrows. This ensures absolute trust, sub-second finality, and transaction fees of less than \$0.00001 per transaction.

This document outlines the core platform features, architectural requirements, actor-role workflows, smart contract parameters, and the on-chain registry structure supporting multi-tenant enterprise environments.

2. Target Audience & Problem Statement

The traditional freelance network model relies heavily on centralized platforms that impose 10%–20% service fees, introduce payment delays of up to 14 business days, and leave contributors vulnerable to arbitrary chargebacks and payment disputes.

2.1 Key Customer Pain Points

- **Payment Insecurity:** Contributors often execute project deliverables without verified funding locks, resulting in payment default or renegotiation.
- **High Processing Overhead:** Cross-border wire transfers and traditional processors consume significant time and cost (averaging 5%–15% in fees).
- **Siloed Platforms:** Contribution histories and reputation rankings are locked inside centralized platform ecosystems, precluding portable worker profiles.

2.2 Target Market

- **Decentralized Autonomous Organizations (DAOs)** needing atomic contributor payouts.
- **Global Tech Enterprises** seeking frictionless cross-border payments.
- **Freelance Developers & Contractors** looking for trustless milestones.

3. Actor Roles & Permissions

The platform models three primary user personas, each interacting with the underlying Soroban contracts and the UI database schema with distinct access rights:

User Role	System Permissions	Key Actions
Creator (Org Admin)	Write access to project creation, escrow financing, and delivery validation.	Bootstraps contracts, creates tasks, funds escrows, verifies deliverables, releases payouts.
Contributor	Write access to registrations, applications, and task completions.	Claims funded tasks, registers professional profile, submits project completion URLs.
Admin (Arbitrator)	Write access to platform governance parameters and dispute overrides.	Sets protocol fees, resolves payment disputes, splits escrowed funds, pauses contract.

4. Functional Specifications

4.1 On-Chain Escrow & Payout System

All financial transactions are governed directly by a Soroban smart contract. The contract guarantees that once a Creator allocates funds to a task, they cannot claw back the funds arbitrarily.

- **Fund Lock:** Escrow deposits lock native XLM, USDC, or EURC assets directly into the contract storage.
- **Platform Fee:** The contract dynamically deducts a customizable basis point fee (default: 2.5%) during payout execution.
- **Instant Payout:** Payout commands release net funds to the contributor's public key

address instantly.

4.2 Interactive Contract Method Explorer

To assist audit compliance and technical users, the landing page includes an interactive Contract Explorer. The explorer lists the 12 primary Soroban functions, accepts test parameters, and simulates transaction envelopes using live workspace Zustand states or web3 wallet injection.

4.3 Multi-Tenant Workspace Isolation

The platform supports isolation for multiple enterprise clients. PostgreSQL Row-Level Security (RLS) is applied to keep tenant data separate. Users belonging to Tenant A cannot view employee databases, ledger sheets, or custom keys belonging to Tenant B.

5. Soroban Smart Contract Architecture

The contract exposes 12 public entrypoints. All mutations require authorization signature checks via `address.require_auth()`.

5.1 Method Index & Specifications

```
1. bootstrap(admin: Address, fee_bps: u32, token: Address, fee_recipient: Address)
```

- Parameters: Admin identity, platform commission, token address, commission receiver.

- Restrict: Callable only once during initial deployment.

2. `deposit_reward(task_id: Symbol, reward: u64)`

- Parameters: Unique task ID, token amount to transfer.
- Restrict: Requires token transfer allowance from caller.

3. `claim(task_id: Symbol, contributor: Address)`

- Parameters: Task ID, contributor address.
- Restrict: Task must be funded, caller must be assigned assignee.

4. `submit(task_id: Symbol, delivery_url: String)`

- Parameters: Task ID, URL of deliverables.
- Restrict: Caller must be the assigned contributor.

5. `verify(task_id: Symbol)`

- Parameters: Task ID.
- Restrict: Releases net payout to assignee, sends `fee_bps` to `fee_recipient`.

6. `cancel(task_id: Symbol)`

- Parameters: Task ID.
- Restrict: Refunds lock reward to creator. Valid only prior to assignee claiming.

7. `dispute(task_id: Symbol, reason: String)`

- Parameters: Task ID, text explanation.
- Restrict: Freezes escrow; locks release actions until admin resolves.

8. `admin_split(task_id: Symbol, creator_share: u32, assignee_share: u32)`

- Parameters: Task ID, creator share BPS, assignee share BPS.
- Restrict: Authorized only for adminAddress. Sum of shares must equal 10000 BPS.

9. `register_developer(username: String, bio: String)`

- Parameters: Developer username handle, professional bio details.
- Restrict: Registers on-chain profile for sender key.

10. `increment_reputation(developer: Address, amount: u32)`

- Parameters: Developer key address, reputation value to add.
- Restrict: Accessible only upon `verify()` execution.

11. `fetch_task(task_id: Symbol) → TaskStruct`

- Parameters: Task ID.
- Return: Struct containing status, creator, assignee, escrow, reward token.

12. `retrieve_profile(developer: Address) → DeveloperProfile`

- Parameters: Developer address.
- Return: Username, rating score, active contracts list.

6. Non-Functional & Security Requirements

- **Sub-second UI Responsiveness:** Local caching via Zustand and React Query ensures that state mutations are rendered immediately, regardless of network latency.
- **Strict Authorization:** Secure wallet message signing (Freighter/xBull) is mandatory for executing any mutable state functions on the Stellar Testnet.
- **Portable Reputation:** Rep points accumulated on-chain must be stored in the contract's instance storage, allowing integration with external Web3 job boards.